Lista de Exercícios: Orientação a Objetos em Python

UniAcademia, Juiz de Fora/MG · Prof. Marcos Miguel

Orientações gerais

Esta lista tem 10 exercícios relâmpago em ordem crescente de dificuldade, pensados para quem está começando orientação a objetos em Python. Resolva na ordem, porque cada bloco usa o que o anterior ensinou.

Bloco	Tema	Exercícios
1	Classe, objeto e encapsulamento	1 e 2
2	Métodos com regras e estado	3 e 4
3	Trabalhando com datas	5
4	Associação (1 e n)	6 e 7
5	Herança e polimorfismo	8 e 9
Desafio	Integrador	10

Distribuição dos 20 pontos. O valor de cada exercício cresce com a dificuldade.

Exercício	Tema	Pontos
1	Funcionário	1,0
2	IMC	1,5
3	Conta bancária	2,0
4	Controle de estoque simples	2,0
5	Dependentes e aniversários	2,0
6	Cursos, turmas, alunos e professores	2,5
7	Naturalidade e escolaridade	1,5

Exercício	Tema	Pontos
8	Alunos, professores e aprovação	2,0
9	Locadora de veículos	2,5
10	Folha de pagamento (desafio)	3,0
Total		**20,0**

Regras de todos os exercícios

- Crie um único projeto para todos os exercícios, com uma pasta por exercício: `Exercicio1`, `Exercicio2`, até `Exercicio10`. Cada pasta tem o seu próprio `main.py` de execução e uma subpasta `model/` com uma classe por arquivo (por exemplo `Exercicio1/model/Funcionario.py`). No `main.py`, a função `principal()` testa as classes e imprime os resultados, e é chamada dentro de `if __name__ == '__main__':`. Cada `main.py` deve rodar sozinho, sem depender de outro exercício: se precisar de uma classe feita antes (Ex5 e Ex10 reaproveitam `Funcionario` e `Dependente`), copie o arquivo para a pasta do exercício atual.
- Classes em PascalCase e métodos em camelCase (`calcularSalarioLiquido`).
- Atributos privados com dois sublinhados (`self.__nome`), acessados apenas por métodos `getNome()` e `setNome()`. Os métodos `get` e `set` não aparecem nos diagramas: crie conforme a necessidade.
- Métodos que incluem ou retiram elementos de uma lista começam com `add` e `remove`, como `addDependentes` e `removeDependentes`.
- Para exibir um objeto, use `__str__` (o `print` chama sozinho) ou um método `toString()` que devolve a string.
- Os diagramas são ponto de partida: pode acrescentar atributos, métodos e parâmetros que a solução exigir.
- Cada exercício traz um exemplo de entrada e saída. Seu código deve produzir exatamente aquela saída.
- Pode fazer em dupla. Pode consultar material, mas esteja pronto para explicar cada linha que escreveu.
- Entrega em tarefa própria no portal, no prazo combinado em sala. Quanto mais demorar, mais perde ponto.

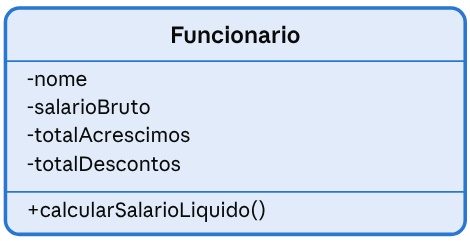
Se travar, releia o enunciado, desenhe o diagrama no papel e só depois volte ao código.

Bloco 1: classe, objeto e encapsulamento

Exercício 1: Funcionário (1,0 ponto)

Objetivo: criar a primeira classe, com construtor, atributos privados, `get` / `set` e um método de cálculo.

Funcionario guarda quatro atributos privados e calcula o salário líquido



Como ler o diagrama

- atributo privado (no Python, nome com prefixo __)
+ método público
getNome() e setNome() dão acesso aos atributos privados

classe Funcionario · 4 atributos, 1 método

1. Crie a classe `Funcionario` (arquivo `model/Funcionario.py`) com os atributos `nome`, `salarioBruto`, `totalAcrescimos` e `totalDescontos`. Os atributos são privados (`self.__nome` e assim por diante) e recebidos pelo construtor.
2. Construa os métodos de acesso de cada atributo: `getNome()` e `setNome()`, `getSalarioBruto()` e `setSalarioBruto()`, e assim por diante.
3. Crie o método `calcularSalarioLiquido()`, que devolve salário bruto mais total de acréscimos menos total de descontos.
4. Na função `principal()` do `main.py`, teste a classe com pelo menos dois funcionários.

Exemplo de entrada e saída:

```
f = Funcionario("Ana", 3000.0, 500.0, 200.0)
print(f.calcularSalarioLiquido())
f.setSalarioBruto(3500.0)
print(f.calcularSalarioLiquido())
```

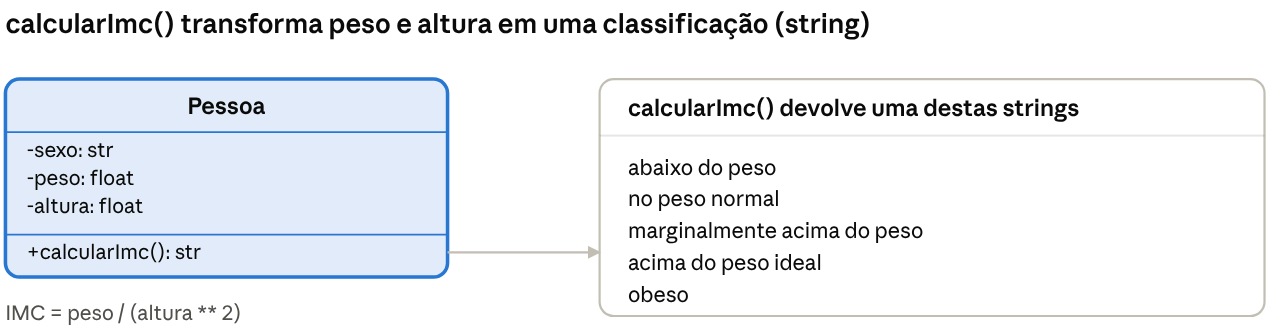
```
3300.0
3800.0
```

Para quem terminar cedo: faça `setSalarioBruto` recusar valores negativos, levantando `ValueError`, e capture o erro em `principal()` com `try/except`. Depois execute `f.nome = 'Outro'` fora da classe e imprima `f.getNome()`. O nome mudou? Explique por quê

(name mangling).

Exercício 2: Índice de Massa Corporal (IMC) (1,5 ponto)

Objetivo: aplicar condicionais dentro de um método e testar com valores limite.



classe Pessoa · IMC e as 5 classificações

O IMC é `peso / (altura ** 2)`. O método `calcularImc()` devolve a **string** da classificação, conforme a tabela:

Condição	IMC em mulheres (`"F"`)	IMC em homens (`"M"`)
abaixo do peso	imc < 19,1	imc < 20,7
no peso normal	19,1 <= imc < 25,8	20,7 <= imc < 26,4
marginalmente acima do peso	25,8 <= imc < 27,3	26,4 <= imc < 27,8
acima do peso ideal	27,3 <= imc < 32,3	27,8 <= imc < 31,1
obeso	imc >= 32,3	imc >= 31,1

1. Implemente a classe `Pessoa` com os atributos privados `sexo`, `peso` e `altura`, exatamente como no diagrama, acrescentando apenas os métodos `get` e `set`.
2. O retorno de `calcularImc()` deve ser uma das cinco strings da coluna "Condição".
3. Construa os casos de teste com o módulo `unittest` (como em `model/Teste.py`), cobrindo cada faixa dos dois sexos e os valores exatos de fronteira (19,1, 25,8 e assim por diante).

Exemplo de entrada e saída:

```
print(Pessoa("F", 60.0, 1.65).calcularImc())
print(Pessoa("M", 100.0, 1.80).calcularImc())
```

```
no peso normal
acima do peso ideal
```

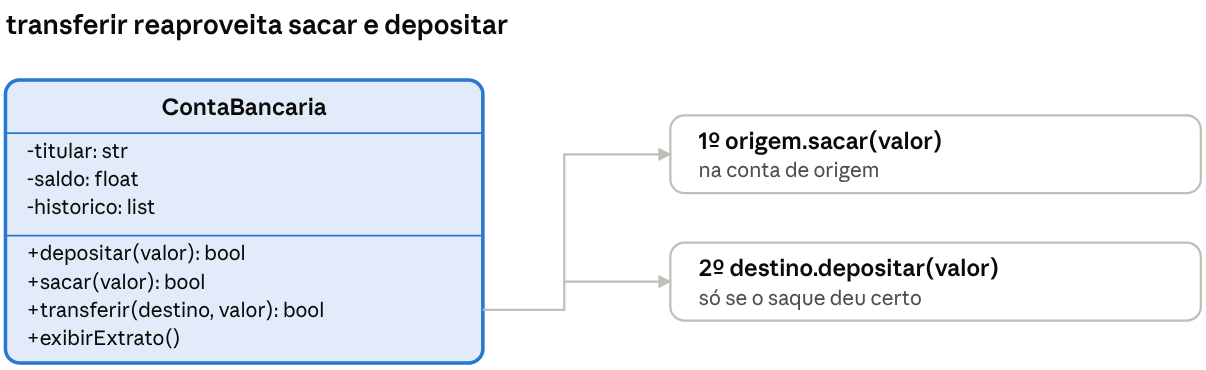
Dica: com `altura = 1.0` o IMC é igual ao peso, o que facilita testar as fronteiras sem erro de arredondamento de `float`.

Bloco 2: métodos com regras e estado

Aqui o objeto passa a "lembrar" do que aconteceu com ele: os métodos mudam o estado e só agem quando a regra permite.

Exercício 3: Conta bancária (2,0 pontos)

Objetivo: validar entradas, devolver `True` / `False` e reaproveitar um método dentro de outro.



classe ContaBancaria · transferir chama sacar e depositar

1. O construtor recebe o titular e o saldo inicial (padrão 0.0), guardados em atributos privados. Crie `getTitular()` e `getSaldo()`, mas não crie `setSaldo()`: o saldo só muda pelos métodos da classe.
2. `depositar`: só aceita valor maior que zero. Se não aceitar, exiba `Valor inválido` e devolva `False`.
3. `sacar`: só executa se houver saldo suficiente. Se não houver, exiba `Saldo insuficiente` e devolva `False`.

4. `transferir`: chama `sacar` na conta de origem e, se der certo, `depositar` na conta de destino. Devolve `True` ou `False` conforme o resultado.
5. Cada operação concluída acrescenta uma linha ao histórico (uma lista privada). Operações recusadas não entram.
6. `exibirExtrato`: mostra o titular, uma linha por operação e o saldo atual.

Exemplo de entrada e saída:

```
c1 = ContaBancaria("Marcos", 100.0)
c2 = ContaBancaria("Ana")
c1.depositar(50.0)
c1.sacar(500.0)
c1.transferir(c2, 80.0)
c1.exibirExtrato()
print(c2.getSaldo())
```

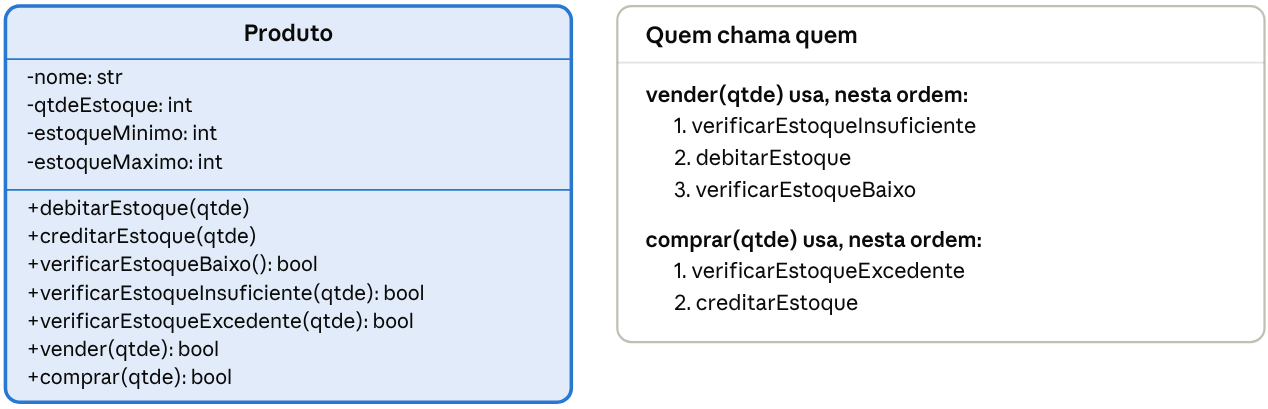
```
Saldo insuficiente
Extrato de Marcos
Depósito de R$ 50.0
Saque de R$ 80.0
Saldo atual: R$ 70.0
80.0
```

Para quem terminar cedo: em vez de devolver `False`, faça `sacar` levantar `ValueError('Saldo insuficiente')` e trate o erro em `principal()` com `try/except/finally`.

Exercício 4: Controle de estoque simples (2,0 pontos)

Objetivo: métodos que consultam o estado (`verificarEstoqueBaixo`, `verificarEstoqueInsuficiente`, `verificarEstoqueExcedente`) e métodos que decidem com base nessas consultas (`vender`, `comprar`).

vender e comprar combinam os métodos verificar e ajustar estoque



classe Produto · 7 métodos e quem chama quem

Comportamento dos métodos:

- `debitarEstoque` diminui o estoque; `creditarEstoque` aumenta o estoque, ambos de acordo com o parâmetro.
- `verificarEstoqueBaixo` devolve `True` se o estoque atual estiver abaixo do mínimo.
- `verificarEstoqueInsuficiente` devolve `True` se a quantidade do parâmetro for maior que o estoque atual.
- `verificarEstoqueExcedente` devolve `True` se a quantidade do parâmetro somada ao estoque atual passar do máximo.
- `vender`: se o estoque for insuficiente, exiba `Estoque insuficiente` e devolva `False`. Caso contrário, debite o estoque; se ele ficar baixo, exiba `Estoque baixo`. Devolva `True`.
- `comprar`: se a quantidade gerar excedente, exiba `Estoque excedente` e devolva `False`. Caso contrário, credite o estoque e devolva `True`.

Exemplo de entrada e saída:

```
p = Produto("Caneta", 100, 20, 200)
p.vender(95)
p.vender(10)
p.comprar(150)
p.comprar(100)
print(p.getQtdeEstoque())
```

```
Estoque baixo
Estoque insuficiente
Estoque excedente
155
```

Para quem terminar cedo: guarde cada venda e compra numa lista privada `historico` e crie `exibirHistorico()`.

Bloco 3: trabalhando com datas

Exercício 5: Dependentes e aniversários (2,0 pontos)

Objetivo: primeiro contato com associação entre duas classes (um funcionário tem vários dependentes) e com o módulo `datetime`. A cola de datas está no apêndice.

Um funcionário tem vários dependentes; cada um calcula seu aniversário



Funcionario 1 para 0..* Dependente

1. Crie `Dependente` com nome e data de nascimento (um objeto `datetime`, criado com `datetime.strptime('10/03/2015', '%d/%m/%Y')`). Crie `Funcionario` com nome e uma lista privada de dependentes, preenchida por `addDependentes`.
2. `listarDependentes` exibe o nome de cada dependente com a data de nascimento no formato `dd/mm/aaaa`.
3. `proximoAniversario` devolve a data do próximo aniversário, que pode ser no ano corrente ou no seguinte, dependendo da data atual. Se o aniversário for hoje, conta como hoje.
4. `diasParaAniversario` devolve quantos dias faltam para essa data.
5. `diaSemanaAniversario` devolve o nome do dia da semana (de segunda-feira a domingo) em que cairá o próximo aniversário.

6. Receba `dataAtual` como parâmetro (um `datetime`). Em `principal()` você passa `datetime.now()` ou uma data fixa para o teste, assim a saída é sempre a mesma.

Exemplo de entrada e saída:

```
from datetime import datetime

hoje = datetime.strptime('21/10/2026', '%d/%m/%Y')
c = Funcionario("Carla")
c.addDependentes(Dependente("Lucas", datetime.strptime('10/03/2015',
'%d/%m/%Y')))
c.addDependentes(Dependente("Júlia", datetime.strptime('21/10/2018',
'%d/%m/%Y')))
c.addDependentes(Dependente("Pedro", datetime.strptime('04/12/2012',
'%d/%m/%Y')))
c.listarDependentes(hoje)
```

```
Dependentes de Carla
Lucas | nascimento: 10/03/2015 | próximo aniversário: 10/03/2027 | faltam 140
dias | Quarta-feira
Júlia | nascimento: 21/10/2018 | próximo aniversário: 21/10/2026 | faltam 0
dias | Quarta-feira
Pedro | nascimento: 04/12/2012 | próximo aniversário: 04/12/2026 | faltam 44
dias | Sexta-feira
```

Para quem terminar cedo: faça a associação nos dois sentidos, como no projeto de aula. O `Dependente` guarda o responsável (`setPessoa` e `getPessoa`) e mostra o nome dele ao ser listado. Depois trate o dependente nascido em 29/02: nos anos não bissextos, comemore em 28/02 e explique no código por que escolheu essa data.

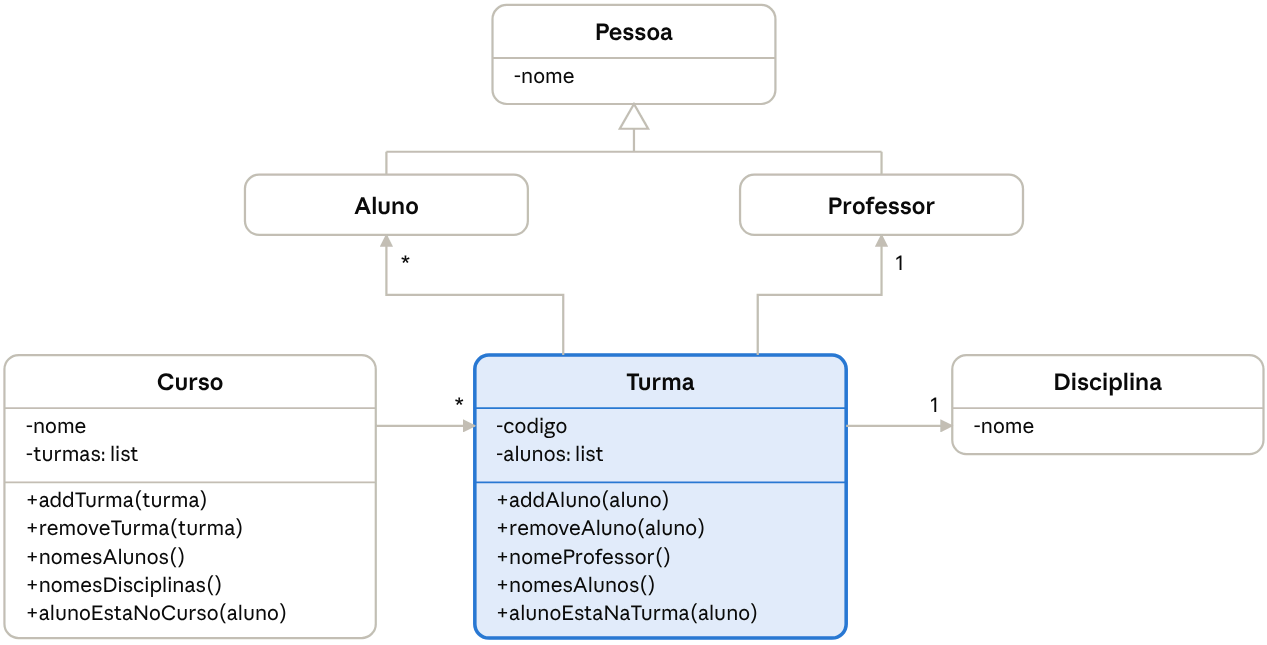
Bloco 4: associação (1 e n)

As setas do diagrama mostram quem conhece quem. Associação do tipo 1 vira um atributo com um objeto; associação do tipo n vira um atributo com uma lista de objetos.

Exercício 6: Cursos, turmas, alunos e professores (2,5 pontos)

Objetivo: percorrer associações 1 e n para responder perguntas, e remover objetos de listas.

Curso reúne turmas; a turma tem um professor, uma disciplina e vários alunos



herança e associações · Curso, Turma, Aluno, Professor, Disciplina

O diagrama tem herança (`Aluno` e `Professor` são `Pessoa`), associação do tipo 1 (turma tem um professor e uma disciplina) e associação do tipo n (curso tem várias turmas; turma tem vários alunos).

Implemente as classes e os métodos abaixo. Os alunos de um curso são os alunos de suas turmas.

1. `Turma.nomeProfessor()` : nome do professor da turma.
2. `Turma.nomesAlunos()` : lista com os nomes dos alunos da turma.
3. `Curso.nomesAlunos()` : nomes dos alunos que estão em alguma turma do curso, sem repetir.
4. `Curso.nomesDisciplinas()` : disciplinas que estão em alguma turma do curso, sem repetir.
5. `Turma.alunoEstaNaTurma(aluno)` e `Curso.alunoEstaNoCurso(aluno)` : devolvem `True` ou `False` .
6. `Turma.removeAluno(aluno)` : tira o aluno da turma.
7. `Curso.removeTurma(turma)` : tira a turma do curso.

Exemplo de entrada e saída:

```
mat = Disciplina("Orientação a Objetos")
prof = Professor("Marcos")
ana, bia, caio = Aluno("Ana"), Aluno("Bia"), Aluno("Caio")

t1 = Turma("T1", prof, mat)
t1.addAluno(ana)
t1.addAluno(bia)
t2 = Turma("T2", prof, mat)
t2.addAluno(bia)
t2.addAluno(caio)

curso = Curso("Sistemas de Informação")
curso.addTurma(t1)
curso.addTurma(t2)

print(t1.nomeProfessor())
print(t1.nomesAlunos())
print(curso.nomesAlunos())
print(curso.nomesDisciplinas())
print(curso.alunoEstaNoCurso(caio))
print(t1.alunoEstaNaTurma(caio))
t1.removeAluno(bia)
print(t1.nomesAlunos())
print(curso.alunoEstaNoCurso(bia))
curso.removeTurma(t2)
print(curso.nomesAlunos())
```

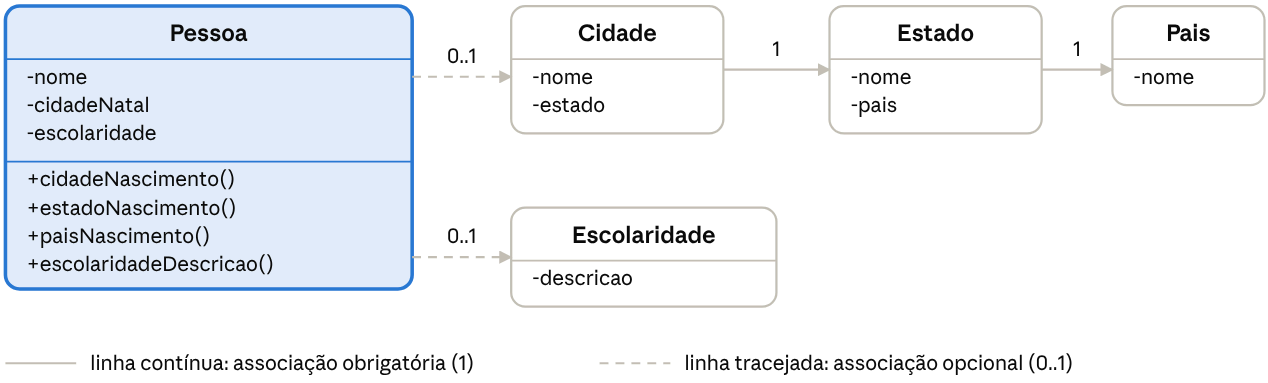
```
Marcos
['Ana', 'Bia']
['Ana', 'Bia', 'Caio']
['Orientação a Objetos']
True
False
['Ana']
True
['Ana']
```

Repare na penúltima linha: a Bia saiu da T1, mas continua no curso porque ainda está na T2.

Exercício 7: Naturalidade e escolaridade (1,5 ponto)

Objetivo: navegar por uma cadeia de associações (pessoa, cidade, estado, país) e decidir o que é obrigatório e o que é opcional.

Pessoa chega a cidade, estado e país pela cadeia de associações



cadeia Pessoa, Cidade, Estado, Pais · obrigatório e opcional

Todas as associações têm multiplicidade 1, mas isso não diz se são obrigatórias. Antes de programar, decida e anote em comentário no código:

- Toda `Cidade` precisa de `Estado`, e todo `Estado` precisa de `Pais` (obrigatórios, passados no construtor).
- A cidade de nascimento e a escolaridade de uma `Pessoa` são opcionais (podem ser `None`).

Implemente, na classe `Pessoa`, métodos que devolvem uma string:

1. `cidadeNascimento()`, `estadoNascimento()` e `paisNascimento()`.
2. `escolaridadeDescricao()`, com a descrição da escolaridade.
3. Quando a informação não existir, o método devolve `Não informado`, sem quebrar o programa.

Exemplo de entrada e saída:

```
br = Pais("Brasil")
mg = Estado("Minas Gerais", br)
jf = Cidade("Juiz de Fora", mg)

ana = Pessoa("Ana", jf)
ana.setEscolaridade(Escolaridade("Superior completo"))
bruno = Pessoa("Bruno")

print(ana.paisNascimento())
print(ana.estadoNascimento())
print(ana.escolaridadeDescricao())
print(bruno.paisNascimento())
```

```
Brasil
Minas Gerais
Superior completo
Não informado
```

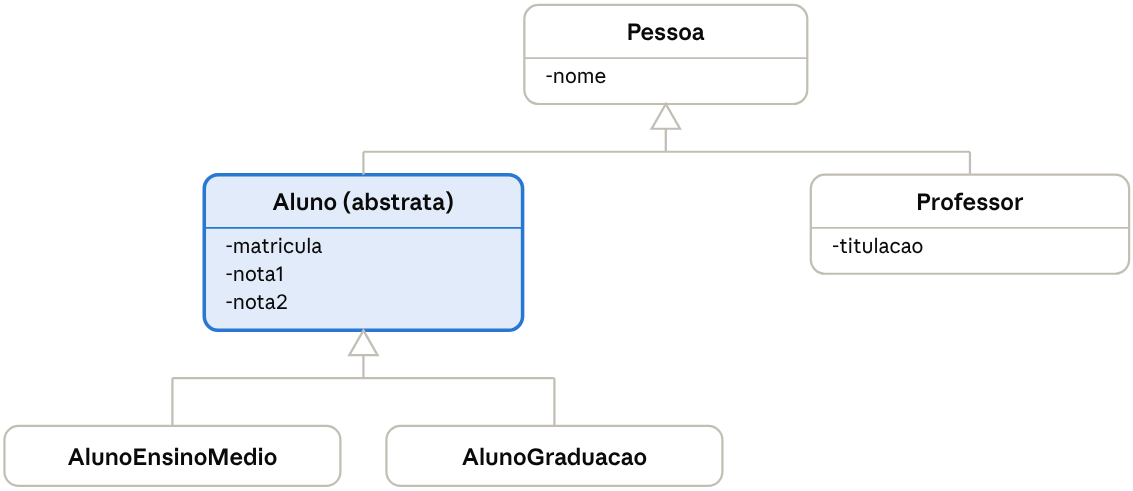
Bloco 5: herança e polimorfismo

A herança evita repetir código: o que é comum fica na classe mãe, e cada filha só escreve o que tem de diferente. Observe que herança não tem multiplicidade.

Exercício 8: Alunos, professores e aprovação (2,0 pontos)

Objetivo: decidir em qual classe cada método deve ficar, usar `super().__init__()` e sobrescrever um método.

Aluno e Professor herdam de Pessoa; os dois tipos de aluno herdam de Aluno



herança · Pessoa, Aluno, Professor e os dois tipos de aluno

Construa as classes e identifique a melhor classe para resolver cada item:

1. `calcularMedia()` : média de duas notas (vale para qualquer aluno).
2. `aprovado()` : no ensino médio, a aprovação ocorre com média maior ou igual a 6; na graduação, com média maior ou igual a 7. Declare `Aluno` como classe abstrata (herdando de `abc.ABC`) e `aprovado()` como método abstrato, como `Pessoa` e `contratar` no projeto de aula. Cada tipo de aluno implementa a sua regra.
3. `situacao()` : devolve a matrícula e o nome do aluno, mais a informação se foi aprovado ou não.
4. `Professor.descricao()` : devolve o nome do professor e sua titulação máxima.

Exemplo de entrada e saída:

```
alunos = [
    AlunoEnsinoMedio("Ana", "2026001", 6.0, 6.0),
    AlunoGraduacao("Bia", "2026002", 6.0, 7.0),
    AlunoGraduacao("Caio", "2026003", 7.0, 8.0),
]
for aluno in alunos:
    print(aluno.situacao())
print(Professor("Marcos", "Mestre").descricao())
```

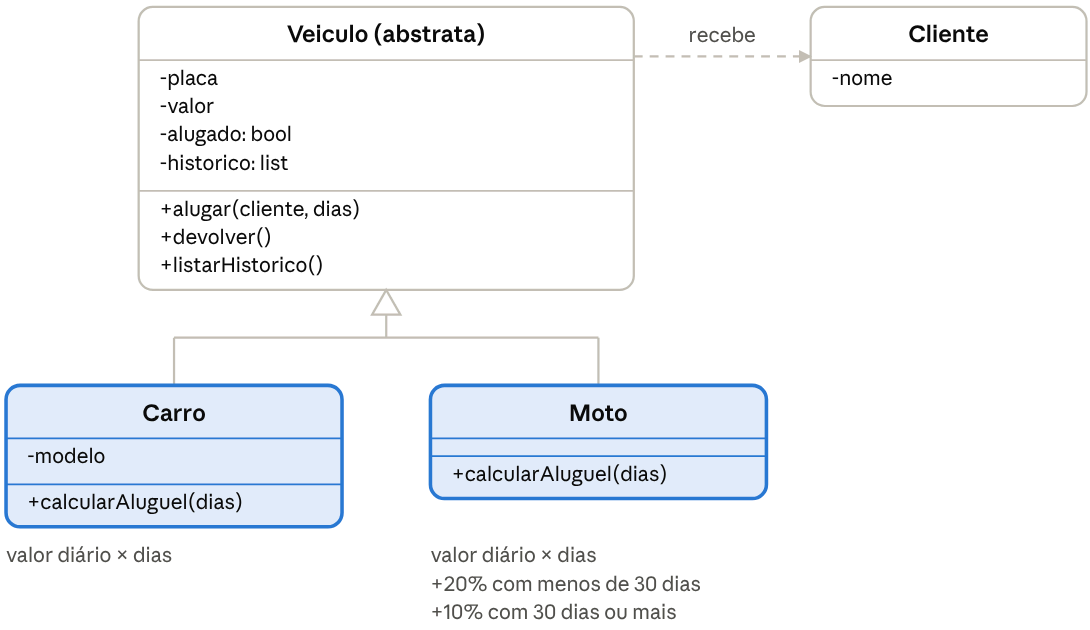
```
2026001 - Ana: aprovado
2026002 - Bia: reprovado
2026003 - Caio: aprovado
Marcos, Mestre
```

Perguntas para discutir: o laço `for` chama `situacao()` sem saber qual é o tipo de cada aluno. Como o Python sabe qual regra de aprovação usar? Esse nome é polimorfismo. E o que acontece se você tentar criar `Aluno(...)` diretamente?

Exercício 9: Locadora de veículos (2,5 pontos)

Objetivo: herança com métodos sobrescritos, controle de estado e histórico.

Carro e Moto herdam de Veiculo e calculam o aluguel cada uma do seu jeito



herança · Veiculo, Carro, Moto e Cliente

Classe `Veiculo`:

- `__alugado` é booleano e indica se o veículo está alugado agora, com `getAlugado()` para consulta. Veículo alugado não pode ser alugado de novo, e veículo não alugado não pode ser devolvido.
- `alugar(cliente, dias)` informa se o aluguel foi confirmado. Se foi, guarda a informação no histórico, com o cliente e o valor.
- `devolver()` informa se a devolução foi feita. Se foi, guarda a informação no histórico.
- Ao alugar ou devolver, atualize o atributo `__alugado`.

Classe `Carro`: o valor do aluguel é o valor diário multiplicado pelo número de dias, calculado em `calcularAluguel(dias)`.

Classe `Moto`: `calcularAluguel(dias)` devolve o valor diário multiplicado pelo número de dias, com acréscimo de 20% se forem menos de 30 dias e de 10% se forem 30 dias ou mais. Arredonde com `round(valor, 2)`.

Exemplo de entrada e saída:

```
rita, paulo = Cliente("Rita"), Cliente("Paulo")
carro = Carro("Onix", "ABC123", 100.0)
moto = Moto("MOT123", 40.0)

carro.alugar(rita, 10)
carro.alugar(rita, 10)
carro.devolver()
carro.devolver()
moto.alugar(paulo, 20)
moto.devolver()
moto.alugar(rita, 30)
carro.listarHistorico()
moto.listarHistorico()
```

```
Aluguel confirmado para o veículo de placa ABC123
Veículo de placa ABC123 não disponível para aluguel
Devolução realizada para o veículo de placa ABC123
Veículo de placa ABC123 não estava alugado
Aluguel confirmado para o veículo de placa MOT123
Devolução realizada para o veículo de placa MOT123
Aluguel confirmado para o veículo de placa MOT123
Histórico do veículo de placa ABC123
Veículo placa ABC123 alugado pelo cliente Rita no valor de R$ 1000.0
Veículo placa ABC123 devolvido
Histórico do veículo de placa MOT123
Veículo placa MOT123 alugado pelo cliente Paulo no valor de R$ 960.0
Veículo placa MOT123 devolvido
Veículo placa MOT123 alugado pelo cliente Rita no valor de R$ 1320.0
```

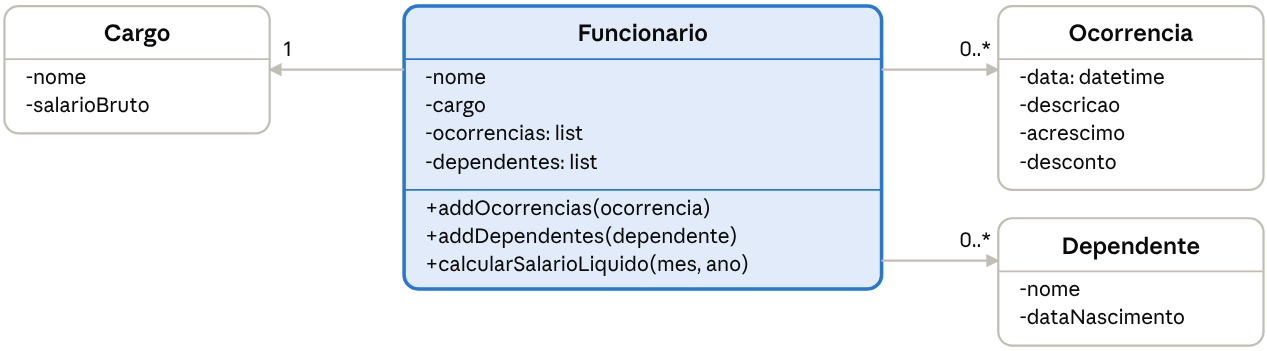
Dica: declare `Veiculo` como classe abstrata (`abc.ABC`) e `calcularAluguel(dias)` como método abstrato. `Veiculo.alugar` chama `self.calcularAluguel(dias)`: a classe mãe não sabe calcular, cada filha sabe. Por isso `calcularAluguel` deve existir nas duas filhas, e não é possível criar um `Veiculo` diretamente.

Desafio integrador

Exercício 10: Folha de pagamento (3,0 pontos)

Objetivo: juntar tudo (encapsulamento, associações 1 e n, datas e métodos com regras) em um único problema. Reaproveite o que fez nos exercícios 1 e 5.

Funcionario reúne cargo, ocorrências e dependentes para o salário líquido



associações · Funcionario, Cargo, Ocorrencia, Dependente

Implemente `Funcionario.calcularSalarioLiquido(mes, ano)` seguindo, nesta ordem:

1. Pegar o salário bruto do cargo daquele funcionário (`getCargo().getSalarioBruto()`).
2. Somar os acréscimos e subtrair os descontos das ocorrências do funcionário (adicionadas com `addOcorrencias`) cujas datas sejam do mês e do ano recebidos. Ocorrências de outros meses são ignoradas.
3. Acrescentar R$ 100,00 para cada dependente (adicionado com `addDependentes`) com menos de 18 anos completos no último dia do mês de cálculo.
4. Devolver o resultado.

Antes de programar, escreva em comentário o que é obrigatório e o que é opcional em cada associação (por exemplo: um funcionário pode ter zero ocorrências e zero dependentes?).

Exemplo de entrada e saída:

```
from datetime import datetime

carla = Funcionario("Carla", Cargo("Analista", 5000.0))
carla.addOcorrencias(Ocorrencia(datetime(2026, 9, 10), "Hora extra",
acrescimo=300.0))
carla.addOcorrencias(Ocorrencia(datetime(2026, 9, 25), "Falta",
desconto=150.0))
carla.addOcorrencias(Ocorrencia(datetime(2026, 10, 5), "Hora extra",
acrescimo=400.0))
carla.addDependentes(Dependente("Lucas", datetime(2015, 3, 10)))
carla.addDependentes(Dependente("Júlia", datetime(2018, 10, 21)))
carla.addDependentes(Dependente("Pedro", datetime(2008, 9, 30)))

print(f"Salário líquido 09/2026: {carla.calcularSalarioLiquido(9, 2026)}")
print(f"Salário líquido 10/2026: {carla.calcularSalarioLiquido(10, 2026)}")
```

```
Salário líquido 09/2026: 5350.0
Salário líquido 10/2026: 5600.0
```

Conferindo a conta de setembro: 5000 + 300 − 150 + 2 × 100. O Pedro faz 18 anos em 30/09/2026, então não entra. Em outubro: 5000 + 400 + 2 × 100.

Dicas: `calendar.monthrange(ano, mes)[1]` devolve o último dia do mês, e com ele você monta `datetime(ano, mes, ultimoDia)`. A idade em anos completos é `ref.year - nasc.year - ((ref.month, ref.day) < (nasc.month, nasc.day))`. Teste também um funcionário sem ocorrências e sem dependentes. Para quem terminar cedo: grave o demonstrativo do mês em um arquivo com `with open`, tratando `OSError` com `try/except/finally`, como no `main.py` de aula.

Apêndice: cola de Python

Classe, encapsulamento e herança

```
import abc


class Pessoa(abc.ABC):                  # classe abstrata
    def __init__(self, nome):
        self.__nome = nome              # __ torna o atributo privado
```

```
    def getNome(self):
        return self.__nome

    def setNome(self, nome):
        self.__nome = nome

    @abc.abstractmethod
    def descricao(self):              # cada filha implementa a sua versão
        pass

    def __str__(self):                # o que print(pessoa) mostra
        return self.__nome


class Aluno(Pessoa):                  # herança
    def __init__(self, nome, matricula):
        super().__init__(nome)        # chama o construtor da classe mãe
        self.__matricula = matricula

    def getMatricula(self):
        return self.__matricula

    def descricao(self):
        return f"{self.getNome()} ({self.__matricula})"
```

Associação e listas de objetos

```
class Turma:
    def __init__(self, codigo, professor):
        self.__codigo = codigo
        self.__professor = professor   # associação 1: um objeto
        self.__alunos = []             # associação n: lista de objetos

    def addAluno(self, aluno):
        self.__alunos.append(aluno)

    def removeAluno(self, aluno):
        self.__alunos.remove(aluno)

    def nomesAlunos(self):
        return [a.getNome() for a in self.__alunos]
```

Datas

```
from datetime import datetime, timedelta

hoje = datetime.now()
print(hoje.day, hoje.month, hoje.year)

texto = hoje.strftime("%d/%m/%Y")                          # data para texto
d1 = datetime.strptime("01/12/2026", "%d/%m/%Y")        # texto para data

dias = (datetime.strptime("25/12/2026", "%d/%m/%Y")
        - datetime.strptime("21/10/2026", "%d/%m/%Y")).days   # diferença em
dias
mais45 = hoje + timedelta(days=45)                         # somar dias

# weekday(): segunda = 0 ... domingo = 6
diaSemana = ("Segunda-feira", "Terça-feira", "Quarta-feira", "Quinta-feira",
             "Sexta-feira", "Sábado", "Domingo")
print(diaSemana[hoje.weekday()])
```

Erros e arquivos

```
arquivo = None
try:
    arquivo = open("demonstrativo.txt", "w")
    arquivo.write("Salário líquido: 5350.0\n")
except OSError as erro:
    print("Não foi possível gravar:", erro)
finally:
    if arquivo is not None:
        arquivo.close()

with open("demonstrativo.txt") as f:      # with fecha o arquivo sozinho
    print(f.read())
```